(Software for)
Modeling and Solving Multiobjective Optimization Problems

Prof. Dr. Xavier Gandibleux

Nantes Université, France

July 2026

Organization

Part 1 (9:00-10:30):

Fundamentals

Building a complete workflow: from modeling to solving and analyzing

Part 2 (11:00-12:30):

Case Studies

Applying the workflow to increasingly challenging case studies

Slides, examples, notebooks

Resources online:

`https://github.com/xgandibleux/EURO_MCDM2026/`

Legend of the marks on the slides:

Additional resources provided

Live demonstration

Instruction(s) to be written

Preamble

Why this adventure ?

More than 30 years of research into multi-objective optimisation

$$\min \left\{ Fx : x \in X \right\}$$

2015: Kick-off of the ANR/DFG research project vOpt

Sub-task 2.2: Experimental Analysis and Prototype Development.

- A consortium composed of mathematicians and informaticians
- MOMILP: Multi-Objective Mixed Integer Linear Programming

$$\min \left\{ Cx + Dy : Ax + By = d,\ x \geqslant 0,\ y \in \mathbb{Z} \right\}$$

↳ The project of a multi-objective MILP solver was born!

Why this adventure ?

More than 30 years of research into multi-objective optimisation

$$\min \left\{ Fx : x \in X \right\}$$

2015: Kick-off of the ANR/DFG research project vOpt

Sub-task 2.2: Experimental Analysis and Prototype Development.

- A consortium composed of mathematicians and informaticians
- MOMILP: Multi-Objective Mixed Integer Linear Programming

$$\min \left\{ Cx + Dy : Ax + By = d,\ x \geqslant 0,\ y \in \mathbb{Z} \right\}$$

↳ The project of a multi-objective MILP solver was born!

Purposes of the solver

Usages:

Natural, intuitive use for mathematicians, informaticians, engineers

- for research:
  support and primitives for the development of new algorithms
- for experimentation:
  methods and algorithms for performing numerical experiments
- for teaching:
  environment for practicing of theories and algorithms

Expectations:

Easy to formulate a problem, provide the data, solve a problem, collect the outputs, analyze the solutions...

+ Solver: robust, efficient, evolutive
+ Software: free, open source, multi-platform

...and easy to install: no need of being expert in computer science!

Purposes of the solver

Usages:

Natural, intuitive use for mathematicians, informaticians, engineers

- for research:
  support and primitives for the development of new algorithms
- for experimentation:
  methods and algorithms for performing numerical experiments
- for teaching:
  environment for practicing of theories and algorithms

Expectations:

Easy to formulate a problem, provide the data, solve a problem, collect the outputs, analyze the solutions...

+ Solver: robust, efficient, evolutive
+ Software: free, open source, multi-platform

...and easy to install: no need of being expert in computer science!

Survey

Raise you hand if you are familiar with

-
-
-

Survey

Raise you hand if you are familiar with

- Matlab, Scilab or Mathematica
-
-

Survey

Raise you hand if you are familiar with

- Matlab, Scilab or Mathematica
- Python, or R
-

Survey

Raise you hand if you are familiar with

- Matlab, Scilab or Mathematica
- Python, or R
- Fortran, C, or C++

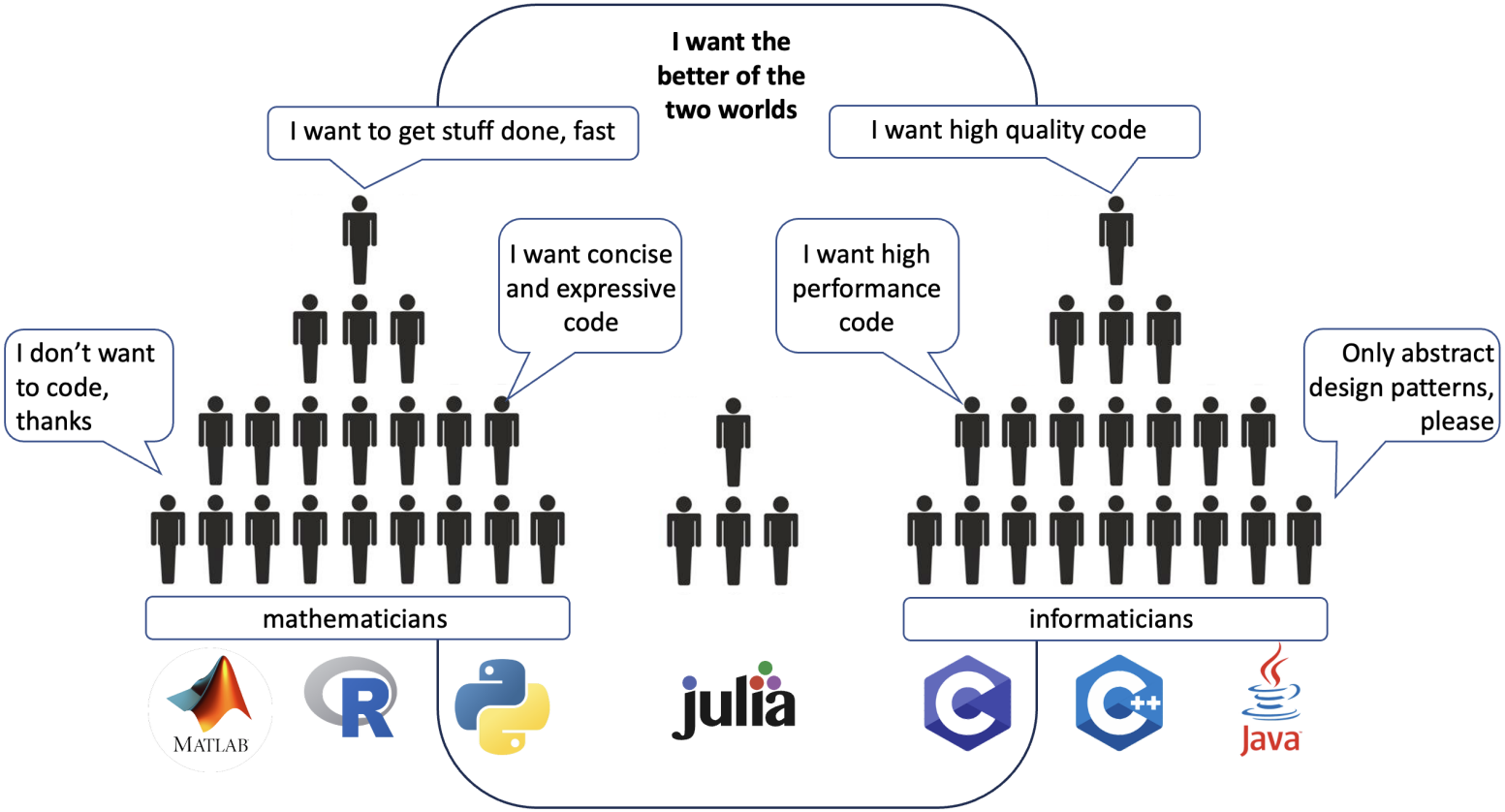
Survey

Raise you hand if you are familiar with

- Matlab, Scilab or Mathematica
- Python, or R
- Fortran, C, or C++

Maths formalisms as Matlab
Interactive as Python
Fast as C/C++

Why Julia and JuMP for MOO?



↳ Julia progr. language: easy, high-level, efficient, expressive

JuMP modeling language: expressive, efficient, evolutive

High-level and expressive?

C

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int A[100][100][100];
    int i,j,k,loop;
    for (loop=0; loop <1000; loop++){
        for (i=0; i<100; i++){
            for (j=0; j<100; j++){
                for (k=0; k<100; k++){
                    A[i][j][k] = loop;
                }
            }
        }
        printf("%d\n", A[99][99][99]);
    }
    return 0;
}
```

Julia

```julia
@time begin

    A = zeros(100,100,100)
    for loop = 1:1000
        for i = 1:100
            for j = 1:100
                for k = 1:100
                    A[i,j,k] = loop
                end
            end
        end
        println(A[100,100,100])
    end

end
```

MATLAB

```matlab
tic

    A = zeros(100,100,100);
    for loop = 1:1000
        for i = 1:100
            for j = 1:100
                for k = 1:100
                    A(i,j,k) = loop;
                end
            end
        end
        disp(A(100,100,100));
    end

toc
```

Efficient?

- C

  without optimization option

  `1.286582 seconds`

  with optimization option `-O3`

  `0.087682 seconds`

- Julia

  1-to-1 straightforward implementation

  `26.732819 seconds (489.05 M allocations: 7.297 GiB, 0.29% gc time)`

  julia style compliant implementation

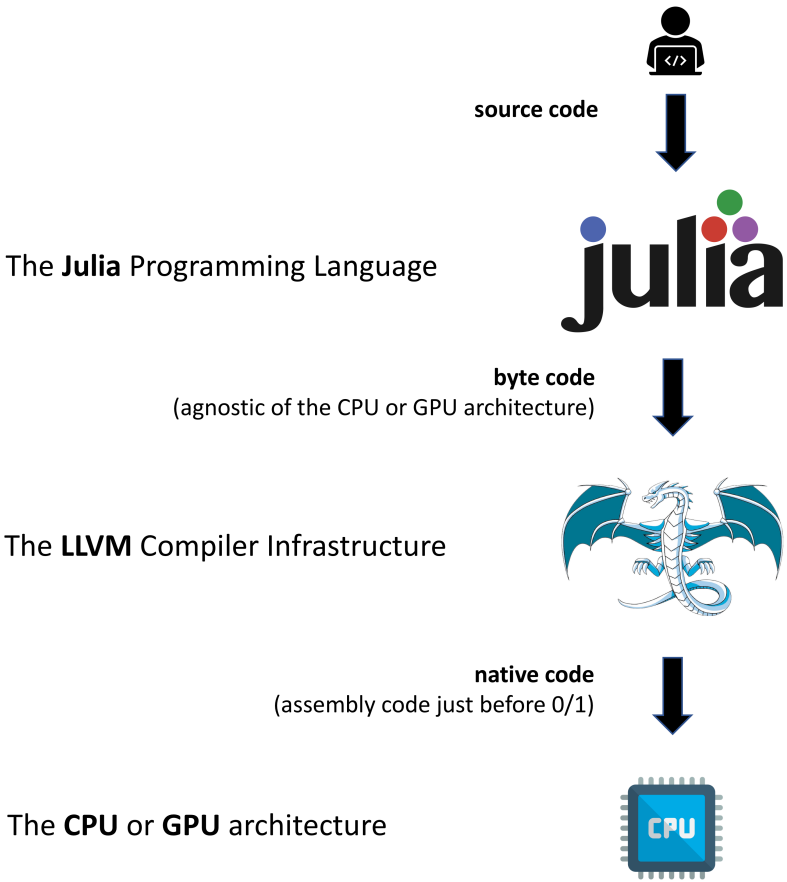
  `0.252995 seconds (22.39 k allocations: 8.499 MiB)`

  polished implementation

  `0.114872 seconds (20.75 k allocations: 4.318 MiB)`

Efficient! Why?

LLVM: middleman between source code and compiled native code




```
julia> 1+2
3
```

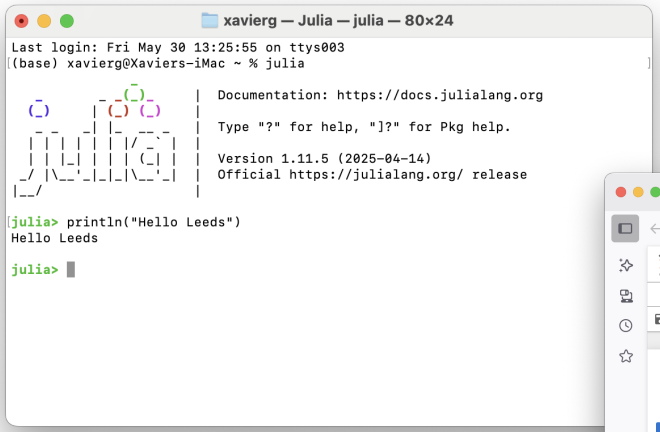
```
julia> @code_llvm 1+2
;  @ int.jl:87 within `+`
define i64 @"julia_+_115"(i64 signext %0, i64 signext %1) #0 {
top:
  %2 = add i64 %1, %0
  ret i64 %2
}
```

```
julia> @code_native 1+2
        .section        __TEXT,__text,regular,pure_instructions
        .build_version macos, 13, 0
        .globl  "_julia_+_150"                  ; -- Begin function julia_+_150
        .p2align        2
"_julia_+_150":                         ; @"julia_+_150"
; ┌ @ int.jl:87 within `+`
        .cfi_startproc
; %bb.0:                                ; %top
        add     x0, x1, x0
        ret
        .cfi_endproc
; └
                                        ; -- End function
.subsections_via_symbols
```

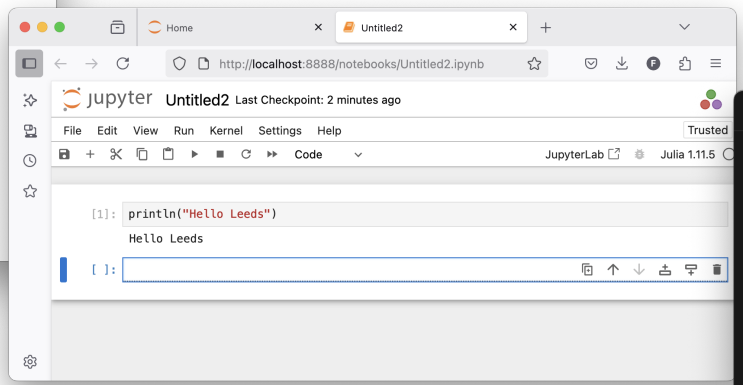
```
01101100
01101111
01110110
01100101
```

Easy to use?

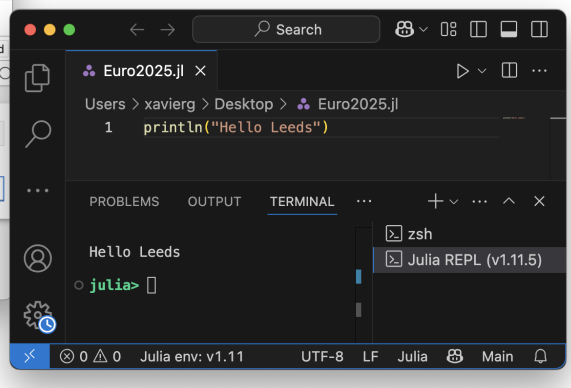
...the Julia REPL



...a notebook
(e.g. jupyter)



...an IDE
(e.g. VScode)



... or also in command line into a terminal.

Julia in a nutshell...

Main key concepts:

- free, open-source, multi-platform
- Julia has an LLVM-based Just-in-time compilation (JIT)
- Compilation down to native code (running on CPU and GPU, ...)
- Julia is dynamically typed, provides multiple dispatch
- Julia has many built-in mathematical functions
- A built-in package manager
- Designed for parallelism and distributed computation

Resources:

The Julia Programming Language (`https://julialang.org/`): download, documentation, resources, community, etc.

Jeff Bezanson, Alan Edelman, Stefan Karpinski and Viral B. Shah. Julia: A Fresh Approach to Numerical Computing. *SIAM Review*, 59: 65–98. 2017.

Julia in a nutshell...

Main key concepts:

- free, open-source, multi-platform
- Julia has an LLVM-based Just-in-time compilation (JIT)
- Compilation down to native code (running on CPU and GPU, ...)
- Julia is dynamically typed, provides multiple dispatch
- Julia has many built-in mathematical functions
- A built-in package manager
- Designed for parallelism and distributed computation

Resources:

The Julia Programming Language (`https://julialang.org/`): download, documentation, resources, community, etc.

Jeff Bezanson, Alan Edelman, Stefan Karpinski and Viral B. Shah. Julia: A Fresh Approach to Numerical Computing. *SIAM Review*, 59: 65–98. 2017.

Julia in a nutshell...

Main key concepts:

- free, open-source, multi-platform
- Julia has an LLVM-based Just-in-time compilation (JIT)
- Compilation down to native code (running on CPU and GPU, ...)
- Julia is dynamically typed, provides multiple dispatch
- Julia has many built-in mathematical functions
- A built-in package manager
- Designed for parallelism and distributed computation

Resources:

The Julia Programming Language (`https://julialang.org/`): download, documentation, resources, community, etc.

Jeff Bezanson, Alan Edelman, Stefan Karpinski and Viral B. Shah. Julia: A Fresh Approach to Numerical Computing. *SIAM Review*, 59: 65–98. 2017.

Multi-Objective Mathematical Programming Software

Multiobjective Optimization Problem (MOP) notations

$$\begin{array}{ll} \min & F(x) \\ s.t. & x \in X \end{array}$$

where:

$$\begin{array}{rcl} X \subseteq \mathbb{R}^n & \longrightarrow & \text{the set of feasible solutions} \\ Y = F(X) \subseteq \mathbb{R}^p & \longrightarrow & \text{the set of images} \end{array}$$

Computing sets X_E and Y_N for MOP:

$$y = F(x)$$

$y^* \in Y$ is nondominated, if $\nexists\, y \in Y$ such that $y_i \leqslant y_i^*,\ \forall i$ and $y \neq y^*$.
Y_N is the set of nondominated points.

$x^* \in X$ is efficient if y^* is nondominated.
X_E is a complete set of efficient solutions.

Multiobjective Optimization Problem (MOP) notations

$$\begin{aligned} \min \quad & F(x) \\ s.t. \quad & x \in X \end{aligned}$$

where:

$$\begin{aligned} X \subseteq \mathbb{R}^n \quad &\longrightarrow \quad \text{the set of feasible solutions} \\ Y = F(X) \subseteq \mathbb{R}^p \quad &\longrightarrow \quad \text{the set of images} \end{aligned}$$

Computing sets X_E and Y_N for MOP:

$$y = F(x)$$

$y^* \in Y$ is nondominated, if $\nexists\, y \in Y$ such that $y_i \leqslant y_i^*,\ \forall i$ and $y \neq y^*$.

Y_N is the set of nondominated points.

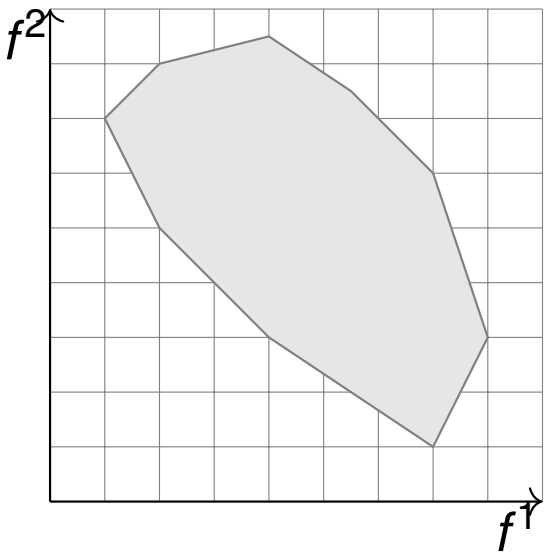
$x^* \in X$ is efficient if y^* is nondominated.

X_E is a complete set of efficient solutions.

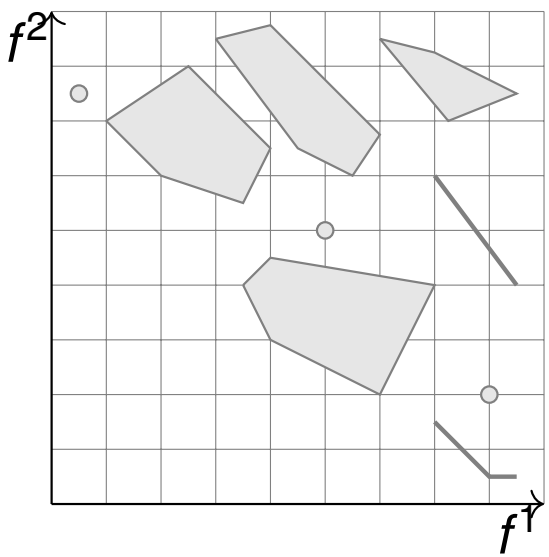
Examples of Y and Y_N when $p = 2$

Computing Y_N for Multiobjective (linear) Optimization Problems (MOP):

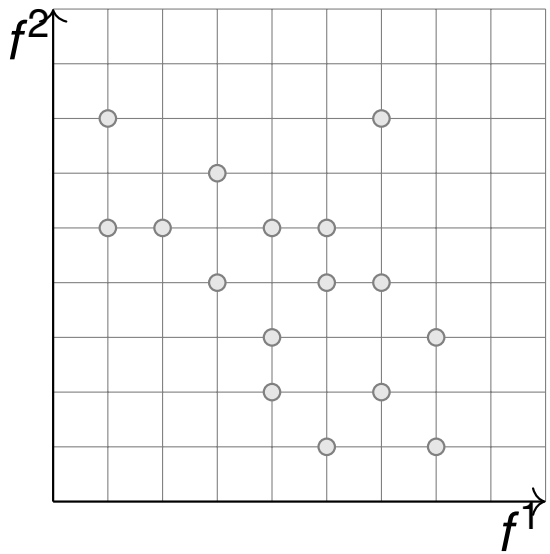
$x \in \mathbb{R}^{n_1}$



$x \in \mathbb{R}^{n_1} \times \mathbb{Z}^{n_2}$



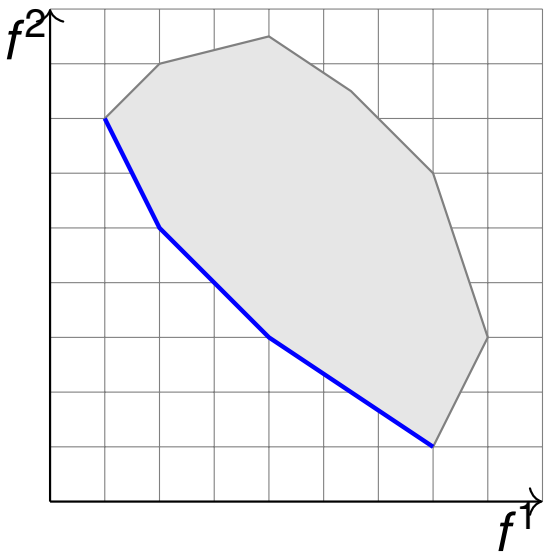
$x \in \mathbb{Z}^{n_2}$



Examples of Y and Y_N when $p = 2$

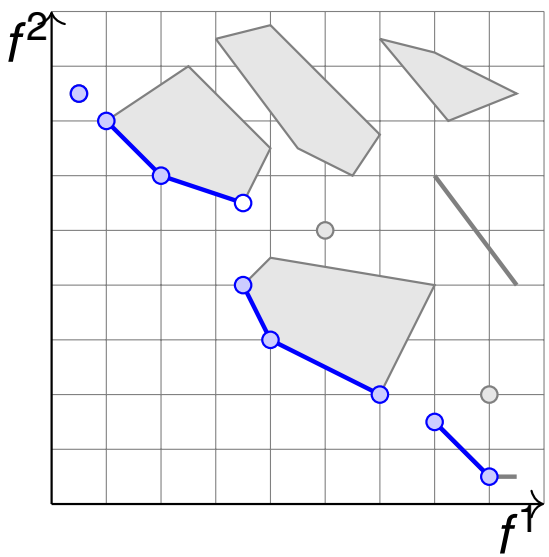
Computing Y_N for Multiobjective (linear) Optimization Problems (MOP):

$x \in \mathbb{R}^{n_1}$



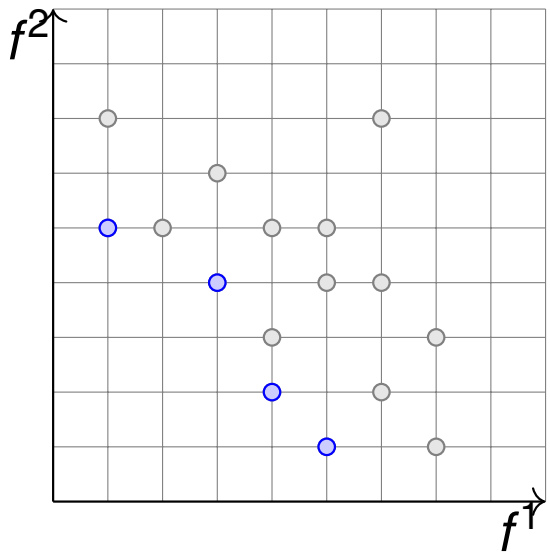
Continuous nondominated set (edges)

$x \in \mathbb{R}^{n_1} \times \mathbb{Z}^{n_2}$



Nondominated set composed of edges that are either closed, half-open, open or reduced to a point

$x \in \mathbb{Z}^{n_2}$

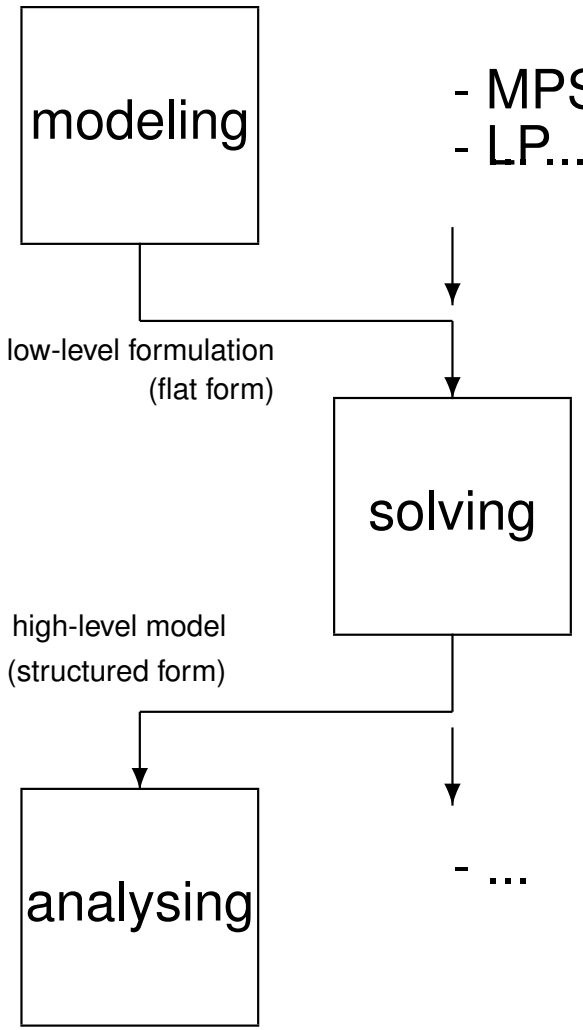


Discrete set of nondominated points

Multi-Objective Mathematical Programming Software

- JuMP
- GNU MP
- Pyomo...

- AMPL
- GAMS
- OPL...



- MPS
- LP...

- HiGHS
- GLPK
- SCIP...

- GUROBI
- CPLEX
- Xpress...

- ...

Software for modeling activity

Modelers: software tools devoted to express mathematical optimisation problems

- Algebraic Modeling Language (AML):

  a high-level programming language designed for mathematical optimisation with an emphasis on declarative modeling and independent of any particular solver.

  - expressed in proprietary languages:
    GAMS, AMPL, AIMMS, OPL, Mosel, GMP, ZIMPL, ...
  - close to programming languages:
    JuMP (Julia), Pyomo (Python),...

- Solver-Oriented Interface (SOI):

  tightly coupled to a specific optimisation solver; provide programmatic control and access to solver-specific features within a host programming language (C/C++, Java, Python,...)

  - Application Programming Interfaces (API):
    API C++ Gurobi,...
  - embedded Domain-Specific Languages (DSL) :
    highspy, MATLAB, Hexaly Modeler,...

Software for modeling activity

Modelers: software tools devoted to express mathematical optimisation problems

- Algebraic Modeling Language (AML):

  a high-level programming language designed for mathematical optimisation with an emphasis on declarative modeling and independent of any particular solver.

  - expressed in proprietary languages:
    GAMS, AMPL, AIMMS, OPL, Mosel, GMP, ZIMPL, ...
  - close to programming languages:
    JuMP (Julia), Pyomo (Python),...

- Solver-Oriented Interface (SOI):

  tightly coupled to a specific optimisation solver; provide programmatic control and access to solver-specific features within a host programming language (C/C++, Java, Python,...)

  - Application Programming Interfaces (API):
    API C++ Gurobi,...
  - embedded Domain-Specific Languages (DSL) :
    highspy, MATLAB, Hexaly Modeler,...

Software for modeling activity

Modelers: software tools devoted to express mathematical optimisation problems

- Algebraic Modeling Language (AML):

  a high-level programming language designed for mathematical optimisation with an emphasis on declarative modeling and independent of any particular solver.

  - expressed in proprietary languages:
    GAMS, AMPL, AIMMS, OPL, Mosel, GMP, ZIMPL, ...
  - close to programming languages:
    JuMP (Julia), Pyomo (Python),...

- Solver-Oriented Interface (SOI):

  tightly coupled to a specific optimisation solver; provide programmatic control and access to solver-specific features within a host programming language (C/C++, Java, Python,...)

  - Application Programming Interfaces (API):
    API C++ Gurobi,...
  - embedded Domain-Specific Languages (DSL) :
    highspy, MATLAB, Hexaly Modeler,...

Software for solving activity

Optimisation solver: a specialized software engine designed to find solutions to mathematical programming problems

- Generic MILP solvers:

  implement numerical algorithms tailored to problem classes (LP, MIP,...)

  - Gurobi, CPLEX, Xpress, MOSEK, Hexaly, COPT,...
  - GLPK, SCIP, HiGHS, CBC/CLP,...

- Specific MOMP solvers:

  specialized optimisation engines designed to directly handle problems with multiple objectives

  - ADBASE, BENSOLVE, INNER, PaMILO, PolySCIP, vOptSpecific

Software for solving activity

Optimisation solver: a specialized software engine designed to find solutions to mathematical programming problems

- Generic MILP solvers:

  implement numerical algorithms tailored to problem classes (LP, MIP,...)

  - Gurobi, CPLEX, Xpress, MOSEK, Hexaly, COPT,...
  - GLPK, SCIP, HiGHS, CBC/CLP,...

- Specific MOMP solvers:

  specialized optimisation engines designed to directly handle problems with multiple objectives

  - ADBASE, BENSOLVE, INNER, PaMILO, PolySCIP, vOptSpecific

Software for solving activity

Optimisation solver: a specialized software engine designed to find solutions to mathematical programming problems

- Generic MILP solvers:

  implement numerical algorithms tailored to problem classes (LP, MIP,...)

  - Gurobi, CPLEX, Xpress, MOSEK, Hexaly, COPT,...
  - GLPK, SCIP, HiGHS, CBC/CLP,...

- Specific MOMP solvers:

  specialized optimisation engines designed to directly handle problems with multiple objectives

  - ADBASE, BENSOLVE, INNER, PaMILO, PolySCIP, vOptSpecific

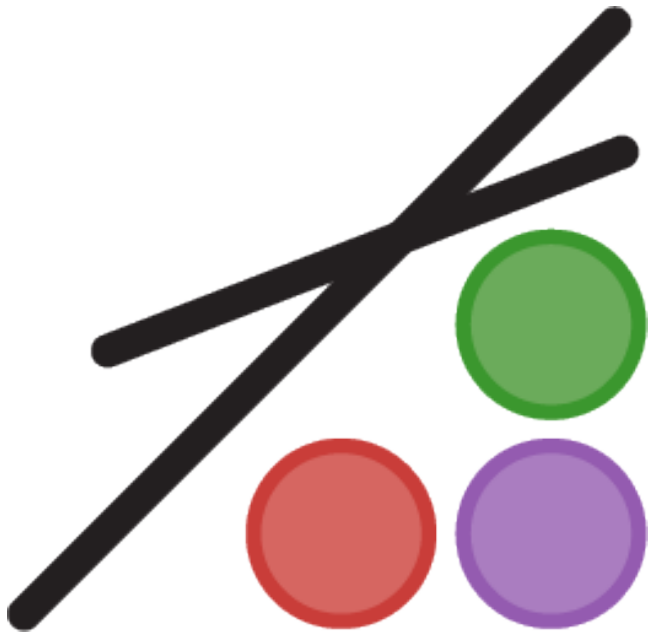
Software for analysing activity

Homemade...

Summary of software examined (Gandibleux 2026)

Software	Commercial	Open-source	Modeler	Solver	# Trade-offs
MATLAB	✓	–	SOI	–	≥1
Gurobi Optimizer	✓	–	SOI	generic	1
HiGHS	–	✓	SOI	generic	1
COPT	✓	–	SOI	generic	1
Hexaly	✓	–	SOI	generic	1
FICO Xpress Optimizer	✓	–	AML+SOI	generic	1
CPLEX Optimisation Studio	✓	–	AML+SOI	generic	1
AMPL	✓	–	AML	–	1
AIMMS	✓	–	AML	–	1
GAMS	✓	–	AML	–	≥1
Pyomo	–	✓	AML	–	–
PuLP	–	✓	AML	–	–
CVXPY	–	✓	AML	specific	1
MOCVXPY	–	✓	AML	specific	≥1
BENSOLVE	–	✓	–	specific	≥1
INNER	–	✓	–	specific	≥1
PaMILO	–	✓	–	specific	≥1
SCIP with ZIMLP and PolySCIP	–	✓	AML	specific	≥1
JuMP with MultiObjectiveAlgorithms	–	✓	AML	meta-solver	≥1

JuMP



version 1.30.0 and more

What is JuMP?

An Algebraic Modeling Language (AML) among the existing ones . . .

... for mathematical optimization (linear, mixed-integer, conic, semidefinite, nonlinear) written in the Julia language.

Overview of JuMP

- **User friendliness**: syntax that mimics natural mathematical expressions.
- **Speed**: similar speeds to special-purpose modeling languages such as AMPL.
- **Solver independence**: Currently 56 solvers are supported.

  Among the (MI)LP solvers: GLPK, Cbc, Clp, HiGHS, SCIP, CPLEX, Gurobi, FICO Xpress, COPT...
- **Ease of embedding**: JuMP itself is written purely in Julia. Solvers are the only binary dependencies.

Overview of JuMP

- **User friendliness**: syntax that mimics natural mathematical expressions.
- **Speed**: similar speeds to special-purpose modeling languages such as AMPL.
- **Solver independence**: Currently 56 solvers are supported.

  Among the (MI)LP solvers: GLPK, Cbc, Clp, HiGHS, SCIP, CPLEX, Gurobi, FICO Xpress, COPT...
- **Ease of embedding**: JuMP itself is written purely in Julia. Solvers are the only binary dependencies.

Overview of JuMP

- **User friendliness**: syntax that mimics natural mathematical expressions.
- **Speed**: similar speeds to special-purpose modeling languages such as AMPL.
- **Solver independence**: Currently 56 solvers are supported.

  Among the (MI)LP solvers: GLPK, Cbc, Clp, HiGHS, SCIP, CPLEX, Gurobi, FICO Xpress, COPT...
- **Ease of embedding**: JuMP itself is written purely in Julia. Solvers are the only binary dependencies.

Overview of JuMP

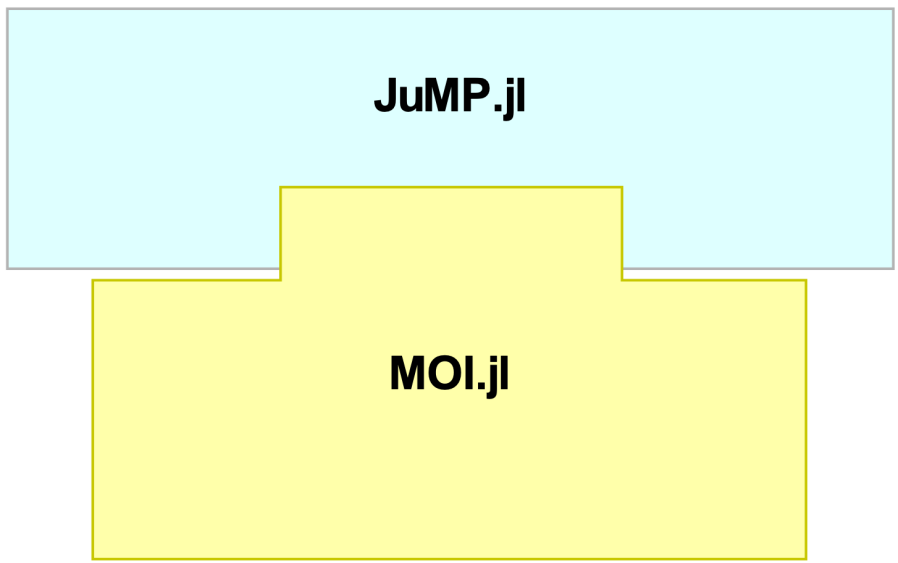
- **User friendliness**: syntax that mimics natural mathematical expressions.
- **Speed**: similar speeds to special-purpose modeling languages such as AMPL.
- **Solver independence**: Currently 56 solvers are supported.

  Among the (MI)LP solvers: GLPK, Cbc, Clp, HiGHS, SCIP, CPLEX, Gurobi, FICO Xpress, COPT...
- **Ease of embedding**: JuMP itself is written purely in Julia. Solvers are the only binary dependencies.

Getting started with JuMP

Getting started

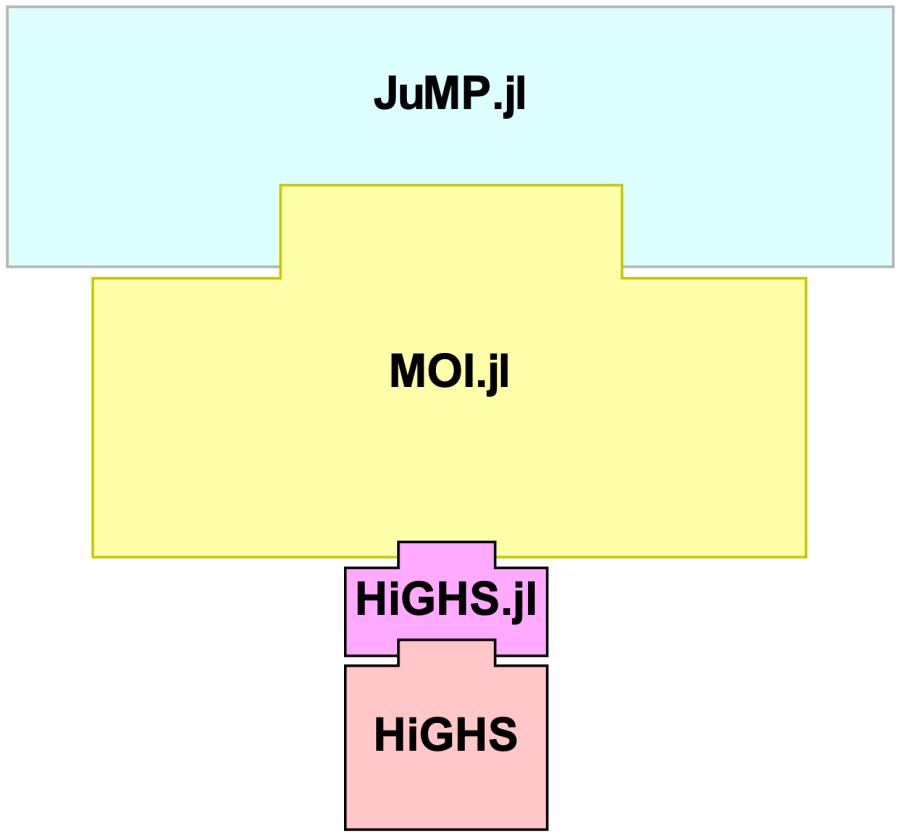
```
julia> using Pkg
julia> Pkg.add("JuMP")
```



JuMP uses *MathOptInterface (MOI)*, an abstraction layer designed to provide a unified interface to mathematical optimization solvers.

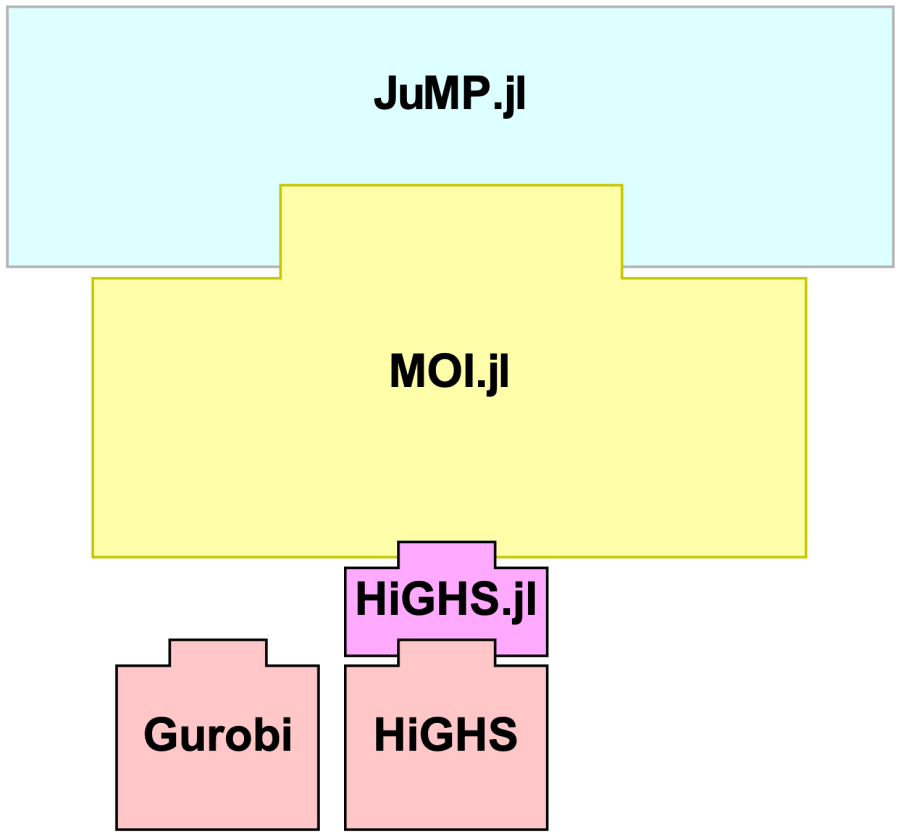
Getting started

```
julia> Pkg.add("HiGHS")
```



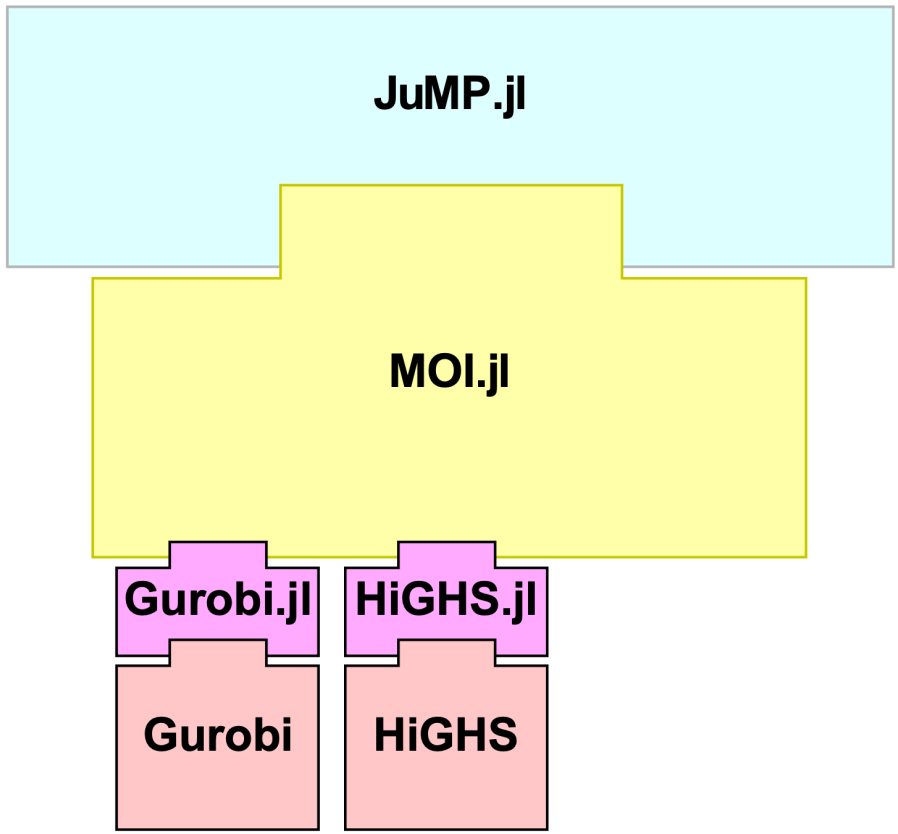
Getting started

With Gurobi or CPLEX or Xpress MP solver properly installed on your computer...



Getting started

```
julia> Pkg.add("Gurobi")
```



Explicit Modeling and Solving of an (MI)LP with JuMP

The optimization problem

$$
\left[\begin{array}{rrrrrrr}
\max z(x) = & x_1 & + & 3x_2 & & & (0) \\
s.t & x_1 & + & x_2 & \leqslant & 14 & (1) \\
& -2x_1 & + & 3x_2 & \leqslant & 12 & (2) \\
& 2x_1 & - & x_2 & \leqslant & 12 & (3) \\
& x_1 & , & x_2 & \geqslant & 0 & (4)
\end{array}\right]
$$

Declaring the packages to use

```
julia> using JuMP, HiGHS
```

Coding the optimization problem (1/4)

Creating a model:

```
modelId = Model( )
```

```
julia> modLP = Model( )
```

Result:

$$\begin{bmatrix} \\ \\ \\ \end{bmatrix}$$

Coding the optimization problem (2/4)

Defining a variable:

```
@variable(modelId, variable)
```

```
julia> @variable(modLP, x1 >= 0)
julia> @variable(modLP, x2 >= 0)
```

Result:

$$\left[\qquad x_1 \quad , \quad x_2 \quad \geqslant \quad 0 \qquad (4) \right]$$

Coding the optimization problem (3/4)

Defining an objective function:

```
@objective(modelId, sense, expression)
```

```
julia> @objective(modLP, Max, x1 + 3x2)
```

Result:

$$\left[\begin{array}{lllll} \max z(x) = & x_1 + 3x_2 & & & (0) \\ & & & & \\ & x_1 , x_2 & \geqslant & 0 & (4) \end{array}\right]$$

Coding the optimization problem (4/4)

Defining a constraint:

```
@constraint(modelId, id, constraint)
```

```
julia> @constraint(modLP, cst1, x1 + x2 <= 14)
julia> @constraint(modLP, cst2, -2x1 + 3x2 <= 12)
julia> @constraint(modLP, cst3, 2x1 - x2 <= 12)
```

Result:

$$\left[\begin{array}{rrrrrrr} \max z(x) = & x_1 & + & 3x_2 & & & (0) \\ s.t & x_1 & + & x_2 & \leqslant & 14 & (1) \\ & -2x_1 & + & 3x_2 & \leqslant & 12 & (2) \\ & 2x_1 & - & x_2 & \leqslant & 12 & (3) \\ & x_1 & , & x_2 & \geqslant & 0 & (4) \end{array}\right]$$

Solving the optimization problem

```
set_optimizer(modelId, solver )
```

```
julia> set_optimizer(modLP, HiGHS.Optimizer)
```

```
set_silent(modelId )
```

```
julia> set_silent(modLP)
```

```
optimize!(modelId)
```

```
julia> optimize!(modLP)
```

Solving the optimization problem

```
set_optimizer(modelId, solver )
```

```
julia> set_optimizer(modLP, HiGHS.Optimizer)
```

```
set_silent(modelId )
```

```
julia> set_silent(modLP)
```

```
optimize!(modelId)
```

```
julia> optimize!(modLP)
```

Solving the optimization problem

```
set_optimizer(modelId, solver )
```

```
julia> set_optimizer(modLP, HiGHS.Optimizer)
```

```
set_silent(modelId )
```

```
julia> set_silent(modLP)
```

```
optimize!(modelId)
```

```
julia> optimize!(modLP)
```

Querying the result

```
julia> @show is_solved_and_feasible(modLP)
```

```
julia> @show termination_status(modLP)
```

```
julia> @show objective_value(modLP)
```

```
julia> @show value(x1)
julia> @show value(x2)
```

Querying the result

```
julia> @show is_solved_and_feasible(modLP)
```

```
julia> @show termination_status(modLP)
```

```
julia> @show objective_value(modLP)
```

```
julia> @show value(x1)
julia> @show value(x2)
```

Querying the result

```
julia> @show is_solved_and_feasible(modLP)
```

```
julia> @show termination_status(modLP)
```

```
julia> @show objective_value(modLP)
```

```
julia> @show value(x1)
julia> @show value(x2)
```

Summary (explicit model)

```
julia> using JuMP, HiGHS

julia> modLP = Model( )
julia> @variable(modLP, x1 >= 0)
julia> @variable(modLP, x2 >= 0)
julia> @objective(modLP, Max, x1 + 3x2)
julia> @constraint(modLP, cst1, x1 + x2 <= 14)
julia> @constraint(modLP, cst2, -2x1 + 3x2 <= 12)
julia> @constraint(modLP, cst3, 2x1 - x2 <= 12)

julia> set_optimizer(modLP, HiGHS.Optimizer)
julia> set_silent(modLP)
julia> optimize!(modLP)

julia> @show is_solved_and_feasible(modLP)
julia> @show objective_value(modLP)
julia> @show value(x1), value(x2)
```

↳ https://github.com/xgandibleux/EURO_MCDM2026/blob/main/3.examples/1LP.ipynb

Details on

Variables:

- by default, the variables are continous and unbounded:

```
julia> @variable(model, x)                  # x is free
```

- possible to setup lower and/or upper bounds on a variable:

```
julia> @variable(model, x≥lb))    # x is bounded
julia> @variable(model, x≤ub)
julia> @variable(model, lb≤x≤ub)
julia> @variable(model, x==2)          # x is fixed
```

- possible to specify the type of a variable:

```
julia> @variable(model, x≥0, Int)            # x ∈ ℕ
julia> @variable(model, x, Bin)           # x ∈ {0,1}
```

Details on

Variables:

- by default, the variables are continous and unbounded:

```
julia> @variable(model, x)                  # x is free
```

- possible to setup lower and/or upper bounds on a variable:

```
julia> @variable(model, x≥lb))    # x is bounded
julia> @variable(model, x≤ub)
julia> @variable(model, lb≤x≤ub)
julia> @variable(model, x==2)       # x is fixed
```

- possible to specify the type of a variable:

```
julia> @variable(model, x≥0, Int)          # x ∈ ℕ
julia> @variable(model, x, Bin)          # x ∈ {0,1}
```

Details on

Variables:

- by default, the variables are continous and unbounded:

```
julia> @variable(model, x)                  # x is free
```

- possible to setup lower and/or upper bounds on a variable:

```
julia> @variable(model, x≥lb))    # x is bounded
julia> @variable(model, x≤ub)
julia> @variable(model, lb≤x≤ub)
julia> @variable(model, x==2)          # x is fixed
```

- possible to specify the type of a variable:

```
julia> @variable(model, x≥0, Int)            # x ∈ ℕ
julia> @variable(model, x, Bin)           # x ∈ {0,1}
```

Implicit Modeling and Solving of an (MI)LP with JuMP

Writing an implicit model (1/5)

Example (cont'd):

$$\left[\begin{array}{rrrcrl} \max f(x) = & x_1 & + & 3x_2 & & (0) \\ s.t & x_1 & + & x_2 & \leqslant 14 & (1) \\ & -2x_1 & + & 3x_2 & \leqslant 12 & (2) \\ & 2x_1 & - & x_2 & \leqslant 12 & (3) \\ & x_1 & , & x_2 & \geqslant 0 & (4) \end{array}\right]$$

$$\downarrow$$

$$c = (1,3),\ d = (14,12,12),\ T = \begin{pmatrix} 1 & 1 \\ -2 & 3 \\ 2 & -1 \end{pmatrix}$$

$$\left[\begin{array}{lll} \max f(x) = & \sum_{j=1}^{2} c_j x_j & (0) \\ s.t & \sum_{j=1}^{2} t_{ij} x_j \leqslant d_i \quad i = 1,3 & (1-3) \\ & x_j \geqslant 0 \qquad\qquad j = 1,2 & (4) \end{array}\right]$$

Writing an implicit model (1/5)


Example (cont'd):

$$c = (1, 3), \quad d = (14, 12, 12), \quad T = \begin{pmatrix} 1 & 1 \\ -2 & 3 \\ 2 & -1 \end{pmatrix}$$

```
julia> c = [1, 3]
julia> d = [14, 12, 12]
julia> T = [1 1 ; -2 3; 2 -1]
julia> n,m = size(T)
```

Writing an implicit model (2/5)

Example (cont'd):

$$\left[\begin{array}{lll} \max z(x) = & \sum_{j=1}^{m} c_j x_j & (0) \\ s.t & \sum_{j=1}^{m} t_{ij} x_j \leqslant d_i \quad i = 1, n & (1-3) \\ & x_j \geqslant 0 \quad j = 1, m & (4) \end{array}\right]$$

```
julia> using JuMP, HiGHS
julia> md = Model(HiGHS.Optimizer)
```

```
julia> @variable(md, x[1:m]⩾0)
```

```
julia> @objective(md, Max,sum(c[j]*x[j] for j=1:m))
```

```
julia> @constraint(md, cst[i=1:n],sum(T[i,j]*x[j]
                                      for j=1:m)⩽d[i])
```

Writing an implicit model (2/5)

Example (cont'd):

$$\left[\begin{array}{lll} \max z(x) = & \sum_{j=1}^{m} c_j x_j & (0) \\ s.t & \sum_{j=1}^{m} t_{ij} x_j \leqslant d_i \quad i = 1, n & (1-3) \\ & x_j \geqslant 0 \quad j = 1, m & (4) \end{array}\right]$$

```
julia> using JuMP, HiGHS
julia> md = Model(HiGHS.Optimizer)
```

```
julia> @variable(md, x[1:m]⩾0)
```

```
julia> @objective(md, Max,sum(c[j]*x[j] for j=1:m))
```

```
julia> @constraint(md, cst[i=1:n],sum(T[i,j]*x[j]
                                      for j=1:m)⩽d[i])
```

Writing an implicit model (2/5)

Example (cont'd):

$$\left[\begin{array}{lll} \max z(x) = & \sum_{j=1}^{m} c_j x_j & (0) \\ s.t & \sum_{j=1}^{m} t_{ij} x_j \leqslant d_i \quad i = 1, n & (1-3) \\ & x_j \geqslant 0 \quad j = 1, m & (4) \end{array}\right]$$

```
julia> using JuMP, HiGHS
julia> md = Model(HiGHS.Optimizer)
```

```
julia> @variable(md, x[1:m]⩾0)
```

```
julia> @objective(md, Max,sum(c[j]*x[j] for j=1:m))
```

```
julia> @constraint(md, cst[i=1:n],sum(T[i,j]*x[j]
                                      for j=1:m)⩽d[i])
```

Writing an implicit model (2/5)

Example (cont'd):

$$\left[\begin{array}{lll} \max z(x) = & \sum_{j=1}^{m} c_j x_j & (0) \\ s.t & \sum_{j=1}^{m} t_{ij} x_j \leqslant d_i \quad i = 1, n & (1-3) \\ & x_j \geqslant 0 \quad j = 1, m & (4) \end{array}\right]$$

```
julia> using JuMP, HiGHS
julia> md = Model(HiGHS.Optimizer)
```

```
julia> @variable(md, x[1:m]⩾0)
```

```
julia> @objective(md, Max,sum(c[j]*x[j] for j=1:m))
```

```
julia> @constraint(md, cst[i=1:n],sum(T[i,j]*x[j]
                                      for j=1:m)⩽d[i])
```

Structured variables in JuMP

- one-based integer ranges...

```
julia> @variable(md, x[1:4]⩾0)
```

```
julia> @variable(md, x[1:2, 1:2]⩾0, Int)
```

- ...but not necessarily...

```
julia> @variable(md, x[-4:4]⩾0)
```

- ...nor necessarily numerical

```
julia> @variable(md, x[[:tram,:train]], Bin)
```

More on solutions

Primal solution values for a scalar variable:

```
value(variable)
```

```
julia> value(x1)
```

Primal solution values for a structured variable:

```
value(variable[index])
```

```
julia> value(x[2])
```

```
value.(variable)
```

```
julia> value.(x)
```

More on solutions

Primal solution values for a scalar variable:

```
value(variable)
```

```
julia> value(x1)
```

Primal solution values for a structured variable:

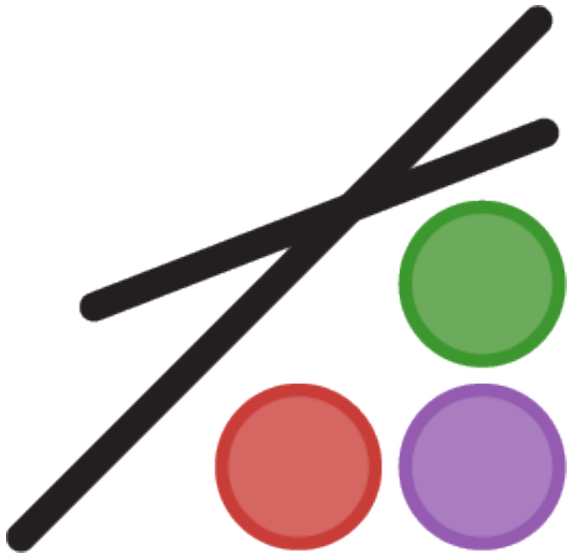
```
value(variable[index])
```

```
julia> value(x[2])
```

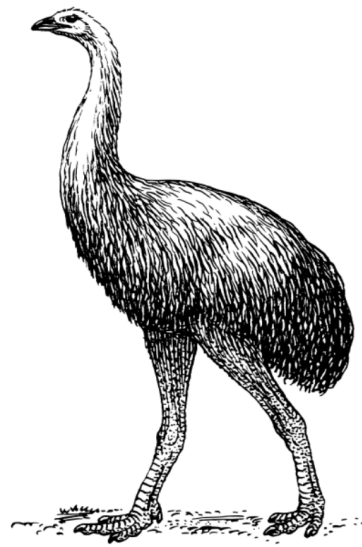
```
value.(variable)
```

```
julia> value.(x)
```

JuMP and MultiObjectiveAlgorithms



version 1.30.0 and more



version 1.12.0 and more

The MOO problem

Compute X_E and Y_N for the following MOO problem:

$$
\begin{array}{lcrcrcr}
\max z_1 & = & x_1 & + & x_2 & & \\
\min z_2 & = & x_1 & + & 3x_2 & & \\
s.t. & & 2x_1 & + & 3x_2 & \leqslant & 30 \\
 & & 3x_1 & + & 2x_2 & \leqslant & 30 \\
 & & x_1 & - & x_2 & \leqslant & 5.5 \\
 & & x_1 & , & x_2 & \in & \mathbb{N}
\end{array}
$$

Getting Started (con't)

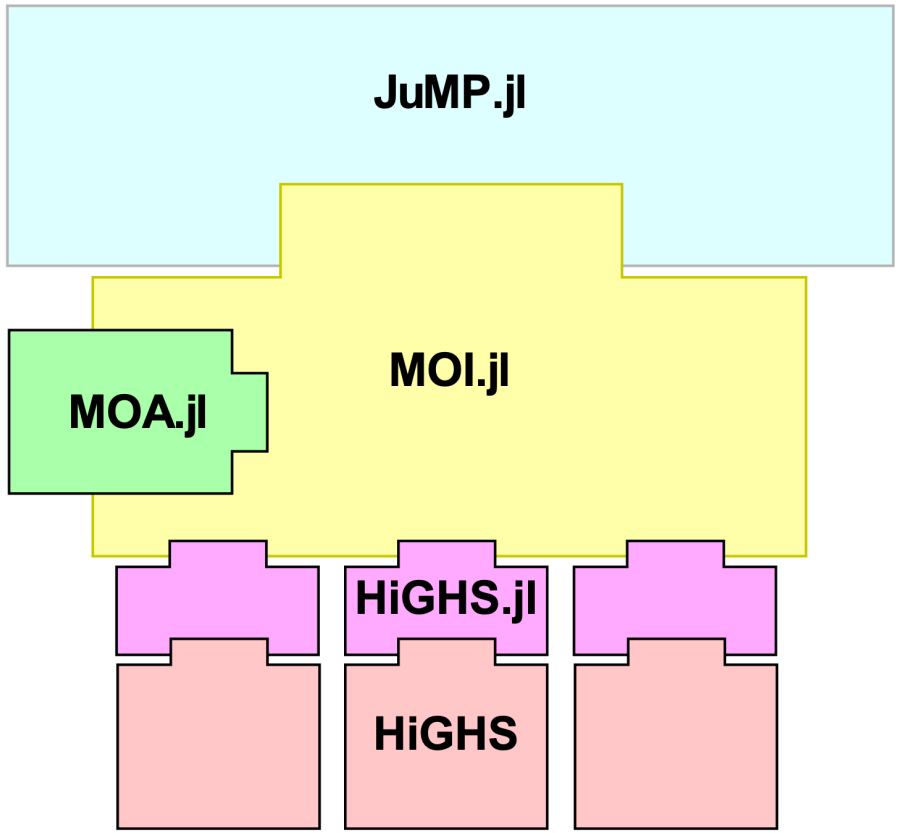
Installing the package:

```
julia> using Pkg
```

```
julia> Pkg.status()
Status `~/.julia/environments/v1.11/Project.toml`
  [60bf3e95] HiGHS v1.18.1
  [4076af6c] JuMP v1.26.0
```

```
julia> Pkg.add("MultiObjectiveAlgorithms")
```

Getting Started (con't)



Algorithms currently available (v1.11.0)

1. MOA.Lexicographic() [default] $p \geqslant 2$
2. MOA.Dichotomy() $p = 2$
3. MOA.EpsilonConstraint() $p = 2$
4. MOA.Hierarchical() $p \geqslant 2$
5. MOA.Chalmet() $p = 2$
6. MOA.KirlikSayin() $p \geqslant 2$
7. MOA.DominguezRios() $p \geqslant 2$
8. MOA.TambyVanderpooten() $p \geqslant 2$
9. MOA.GeneralDichotomy() $p \geqslant 2$

+ optimization attributes coming with a given algorithm

Algorithms currently available (v1.11.0)

1. MOA.Lexicographic() [default] $p \geqslant 2$
2. MOA.Dichotomy() $p = 2$
3. MOA.EpsilonConstraint() $p = 2$
4. MOA.Hierarchical() $p \geqslant 2$
5. MOA.Chalmet() $p = 2$
6. MOA.KirlikSayin() $p \geqslant 2$
7. MOA.DominguezRios() $p \geqslant 2$
8. MOA.TambyVanderpooten() $p \geqslant 2$
9. MOA.GeneralDichotomy() $p \geqslant 2$

+ optimization attributes coming with a given algorithm

Algorithms currently available (v1.11.0)

1. MOA.Lexicographic() [default] $p \geqslant 2$
2. MOA.Dichotomy() $p = 2$
3. MOA.EpsilonConstraint() $p = 2$
4. MOA.Hierarchical() $p \geqslant 2$
5. MOA.Chalmet() $p = 2$
6. MOA.KirlikSayin() $p \geqslant 2$
7. MOA.DominguezRios() $p \geqslant 2$
8. MOA.TambyVanderpooten() $p \geqslant 2$
9. MOA.GeneralDichotomy() $p \geqslant 2$

+ optimization attributes coming with a given algorithm

Zoom on the ϵ-constraint algorithm

Algorithm:

- `EpsilonConstraint()`

  Y.V. Haimes, L.S. Lasdon, D.A. Wismer (1971). On a bicriterion formation of the problems of integrated system identification and system optimization. *IEEE Transactions on Systems, Man and Cybernetics*, Volume SMC-1, Issue 3, Pages 296-297.

Attributes:

- `MOA.EpsilonConstraintStep()`

  algorithm uses this value as the ϵ by which it partitions the first-objective's space. The default is 1, so that for a pure integer program this algorithm will enumerate Y_N.

- `MOA.SolutionLimit()`

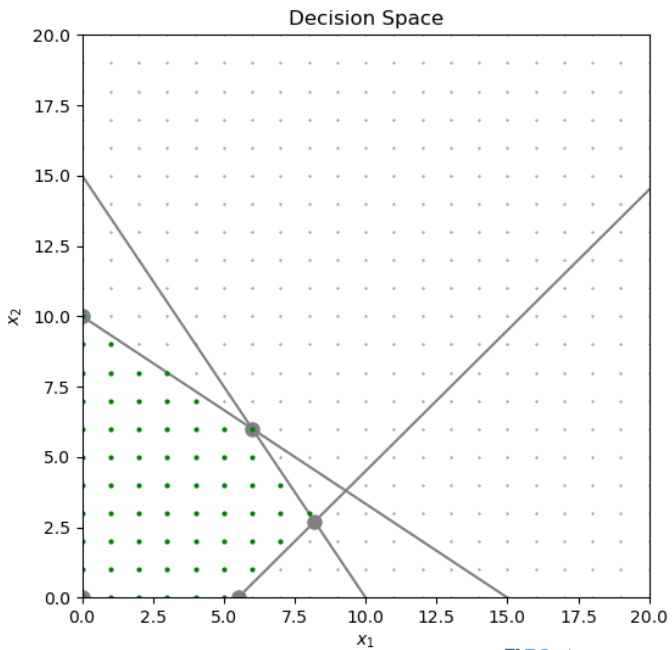
  if this attribute is set then, instead of using the `MOA.EpsilonConstraintStep`, with a slight abuse of notation, `EpsilonConstraint` divides the range of the first-objective's domain in objective space by `SolutionLimit` to obtain the ϵ to use when iterating. Thus, at most `SolutionLimit` solutions are returned.

Example (1/4)

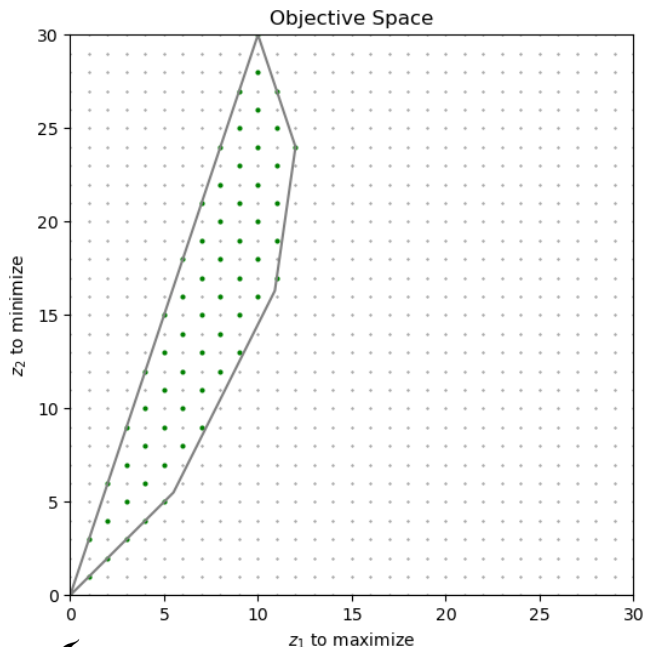
$$
\begin{array}{lcrcrcl}
\max z_1 & = & x_1 & + & x_2 & & \\
\min z_2 & = & x_1 & + & 3x_2 & & \\
 & & & & & & \\
s.t. & & 2x_1 & + & 3x_2 & \leqslant & 30 \\
 & & 3x_1 & + & 2x_2 & \leqslant & 30 \\
 & & x_1 & - & x_2 & \leqslant & 5.5 \\
 & & & & & & \\
 & & x_1 & , & x_2 & \in & \mathbb{N}
\end{array}
$$

```
julia> c1 = [1,1]
julia> c2 = [1,3]
julia> A = [2 3; 3 2; 1 -1]
julia> b = [30, 30, 5.5]
```

Decision space:



Objective space:



Example (2/4)

Coding the MOO problem with JuMP

```
julia> using JuMP, HiGHS
julia> import MultiObjectiveAlgorithms as MOA
```

```
julia> model = Model( )
julia> @variable(model, x1⩾0, Int)
julia> @variable(model, x2⩾0, Int)
julia> @expression(model, fct1, x1 + x2)                    # to maximize
julia> @expression(model, fct2, x1 + 3 * x2)                # to minimize
julia> @objective(model, Max, [fct1, (-1) * fct2])
julia> @constraint(model, 2*x1 + 3*x2 ⩽ 30)
julia> @constraint(model, 3*x1 + 2*x2 ⩽ 30)
julia> @constraint(model, x1 - x2 ⩽ 5.5)
```

Example (2/4)

Coding the MOO problem with JuMP

```
julia> using JuMP, HiGHS
julia> import MultiObjectiveAlgorithms as MOA
```

```
julia> model = Model( )
julia> @variable(model, x1⩾0, Int)
julia> @variable(model, x2⩾0, Int)
julia> @expression(model, fct1, x1 + x2)                    # to maximize
julia> @expression(model, fct2, x1 + 3 * x2)                # to minimize
julia> @objective(model, Max, [fct1, (-1) * fct2])
julia> @constraint(model, 2*x1 + 3*x2 ⩽ 30)
julia> @constraint(model, 3*x1 + 2*x2 ⩽ 30)
julia> @constraint(model, x1 - x2 ⩽ 5.5)
```

Example (3/4)

Setup the MIP solver: e.g. `HiGHS`

```
julia> set_optimizer(model,()->MOA.Optimizer(HiGHS.Optimizer))
```

Setup the algorithm: ϵ-constraint; step=1 (default value)

```
julia> set_attribute(model, MOA.Algorithm(), MOA.EpsilonConstraint())
```

Optimize the MOO problem

```
julia> optimize!(model)
```

Example (3/4)

Setup the MIP solver: e.g. `HiGHS`

```
julia> set_optimizer(model,()->MOA.Optimizer(HiGHS.Optimizer))
```

Setup the algorithm: ϵ-constraint; step=1 (default value)

```
julia> set_attribute(model, MOA.Algorithm(), MOA.EpsilonConstraint())
```

Optimize the MOO problem

```
julia> optimize!(model)
```

Example (3/4)

Setup the MIP solver: e.g. `HiGHS`

```
julia> set_optimizer(model,()->MOA.Optimizer(HiGHS.Optimizer))
```

Setup the algorithm: ϵ-constraint; step=1 (default value)

```
julia> set_attribute(model, MOA.Algorithm(), MOA.EpsilonConstraint())
```

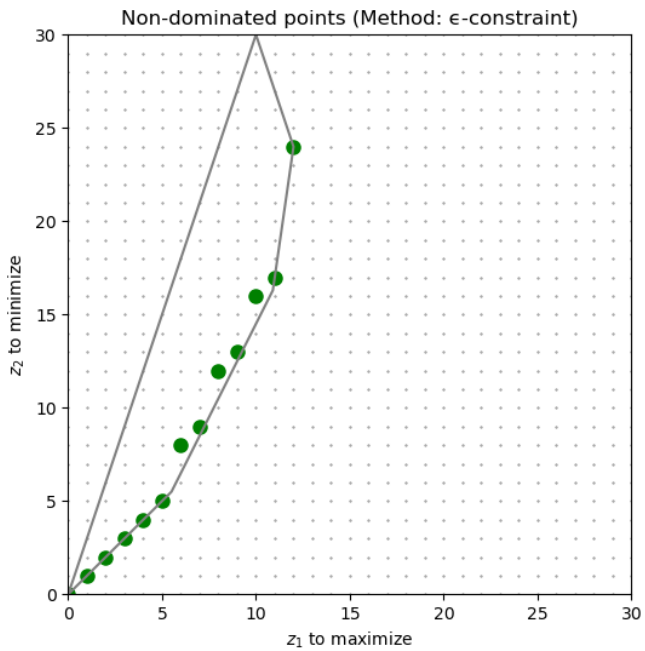
Optimize the MOO problem

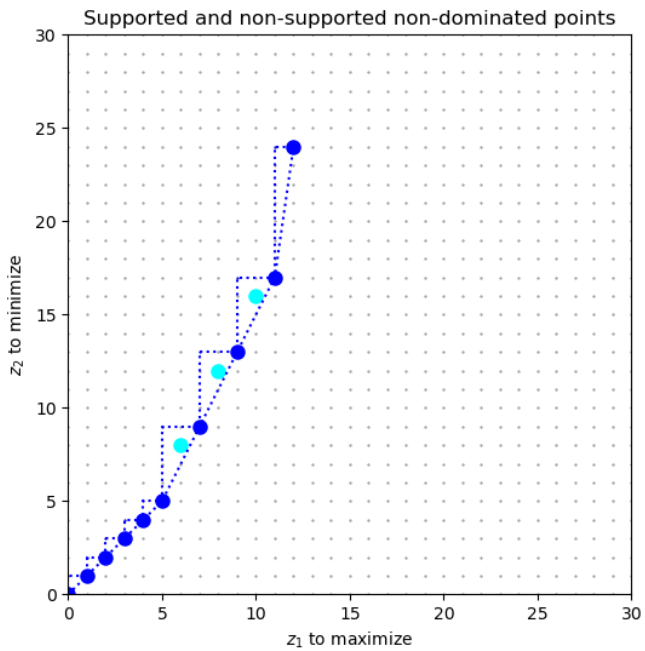
```
julia> optimize!(model)
```

Example (4/4)

Get X_E and Y_N

```
julia> for i in 1:result_count(model)
julia>     z1_opt = objective_value(model; result = i)[1]
julia>     z2_opt = -1 * objective_value(model; result = i)[2]
julia>     x1_opt = value(x1; result = i)
julia>     x2_opt = value(x2; result = i)
julia> end
```





↳ https://github.com/xgandibleux/EURO_MCDM2026/blob/main/3.examples/2IPex.ipynb

Display of Y_N as a figure

```
julia> using Plots
julia> using LaTeXStrings
julia>
julia> Z1opt = [ ]; Z2opt = [ ]
julia> for i in 1:result_count(model)
julia>    push!(Z1opt, objective_value(model; result = i)[1])
julia>    push!(Z2opt, -1 * objective_value(model; result = i)[2])
julia> end
julia>
julia> plot(Z1opt, Z2opt,
julia>    title = "Nondominated points (e-constraint method)",
julia>    xlabel = L"$z_1$ to maximize",
julia>    xlabel = L"$z_2$ to minimize",
julia>    xlims = (0, 30),
julia>    ylims = (0, 30),
julia>    seriestype = :scatter,
julia>    marker = :circle,
julia>    markersize = 5,
julia>    color = :green,
julia>    legend = false
julia> )
```

From
an explicit to an implicit MOO model
with JuMP and MOA

Example (1/3)

$$
\begin{array}{lcrcrcl}
\max z_1 & = & x_1 & + & x_2 & & \\
\min z_2 & = & x_1 & + & 3x_2 & & \\
 & & & & & & \\
s.t. & & 2x_1 & + & 3x_2 & \leqslant & 30 \\
 & & 3x_1 & + & 2x_2 & \leqslant & 30 \\
 & & x_1 & - & x_2 & \leqslant & 5.5 \\
 & & & & & & \\
 & & x_1 & , & x_2 & \in & \mathbb{N}
\end{array}
\quad \rightarrow \quad
\begin{array}{lcll}
\max z_k = & \sum_{j=1}^{n} c_j^k x_j & & k = 1, \ldots, p \\
s.t & \sum_{j=1}^{n} a_{ij} x_j & \leqslant b_i & i = 1, \ldots, m \\
 & x_j & \in \mathbb{N} & j = 1, \ldots, n
\end{array}
$$

```
julia> c1 = [1,1]; c2 = -1 * [1,3]; c = vcat(c1',c2')
julia> a = [2 3; 3 2; 1 -1]
julia> b = [30, 30, 5.5]
julia> m,n = size(a)
julia> p = size(c,1)
```

```
julia> md = Model( )
julia> @variable(md, x[1:n]≥0, Int)
julia> @expression(md, z[k=1:p], sum(c[k,j]*x[j] for j=1:n))
julia> @objective(md, Max, [z[k] for k=1:p])
julia> @constraint(md, ct[i=1:m], sum(a[i,j]*x[j] for j=1:n) ≤ b[i])
```

Example (1/3)

$$
\begin{array}{lcrcrcl}
\max z_1 & = & x_1 & + & x_2 & & \\
\min z_2 & = & x_1 & + & 3x_2 & & \\
 & & & & & & \\
s.t. & & 2x_1 & + & 3x_2 & \leqslant & 30 \\
 & & 3x_1 & + & 2x_2 & \leqslant & 30 \\
 & & x_1 & - & x_2 & \leqslant & 5.5 \\
 & & & & & & \\
 & & x_1 & , & x_2 & \in & \mathbb{N}
\end{array}
\quad \rightarrow \quad
\begin{array}{lcll}
\max z_k = & \sum_{j=1}^{n} c_j^k x_j & & k = 1, \ldots, p \\
s.t & \sum_{j=1}^{n} a_{ij} x_j & \leqslant b_i & i = 1, \ldots, m \\
 & x_j & \in \mathbb{N} & j = 1, \ldots, n
\end{array}
$$

```
julia> c1 = [1,1]; c2 = -1 * [1,3]; c = vcat(c1',c2')
julia> a = [2 3; 3 2; 1 -1]
julia> b = [30, 30, 5.5]
julia> m,n = size(a)
julia> p = size(c,1)
```

```
julia> md = Model( )
julia> @variable(md, x[1:n]⩾0, Int)
julia> @expression(md, z[k=1:p], sum(c[k,j]*x[j] for j=1:n))
julia> @objective(md, Max, [z[k] for k=1:p])
julia> @constraint(md, ct[i=1:m], sum(a[i,j]*x[j] for j=1:n) ⩽ b[i])
```

Example (1/3)

$$
\begin{array}{lcrcrcl}
\max z_1 & = & x_1 & + & x_2 & & \\
\min z_2 & = & x_1 & + & 3x_2 & & \\
s.t. & & 2x_1 & + & 3x_2 & \leqslant & 30 \\
 & & 3x_1 & + & 2x_2 & \leqslant & 30 \\
 & & x_1 & - & x_2 & \leqslant & 5.5 \\
 & & x_1 & , & x_2 & \in & \mathbb{N}
\end{array}
$$

$$
\rightarrow \quad
\begin{array}{lcll}
\max z_k = & \sum_{j=1}^{n} c_j^k x_j & & k = 1, \ldots, p \\
s.t & \sum_{j=1}^{n} a_{ij} x_j & \leqslant b_i & i = 1, \ldots, m \\
 & x_j & \in \mathbb{N} & j = 1, \ldots, n
\end{array}
$$

```
julia> c1 = [1,1]; c2 = -1 * [1,3]; c = vcat(c1',c2')
julia> a = [2 3; 3 2; 1 -1]
julia> b = [30, 30, 5.5]
julia> m,n = size(a)
julia> p = size(c,1)
```

```
julia> md = Model( )
julia> @variable(md, x[1:n]⩾0, Int)
julia> @expression(md, z[k=1:p], sum(c[k,j]*x[j] for j=1:n))
julia> @objective(md, Max, [z[k] for k=1:p])
julia> @constraint(md, ct[i=1:m], sum(a[i,j]*x[j] for j=1:n) ⩽ b[i])
```

Example (1/3)

$$
\begin{array}{lcrcrcl}
\max z_1 & = & x_1 & + & x_2 & & \\
\min z_2 & = & x_1 & + & 3x_2 & & \\
s.t. & & 2x_1 & + & 3x_2 & \leqslant & 30 \\
 & & 3x_1 & + & 2x_2 & \leqslant & 30 \\
 & & x_1 & - & x_2 & \leqslant & 5.5 \\
 & & x_1 & , & x_2 & \in & \mathbb{N}
\end{array}
\quad \rightarrow \quad
\begin{array}{lcll}
\max z_k = & \sum_{j=1}^{n} c_j^k x_j & & k = 1, \ldots, p \\
s.t & \sum_{j=1}^{n} a_{ij} x_j & \leqslant b_i & i = 1, \ldots, m \\
 & x_j & \in \mathbb{N} & j = 1, \ldots, n
\end{array}
$$

```
julia> c1 = [1,1]; c2 = -1 * [1,3]; c = vcat(c1',c2')
julia> a = [2 3; 3 2; 1 -1]
julia> b = [30, 30, 5.5]
julia> m,n = size(a)
julia> p = size(c,1)
```

```
julia> md = Model( )
julia> @variable(md, x[1:n]⩾0, Int)
julia> @expression(md, z[k=1:p], sum(c[k,j]*x[j] for j=1:n))
julia> @objective(md, Max, [z[k] for k=1:p])
julia> @constraint(md, ct[i=1:m], sum(a[i,j]*x[j] for j=1:n) ⩽ b[i])
```

Example (2/3)

Setup and optimize

```
julia> set_optimizer(model, ()->MOA.Optimizer(HiGHS.Optimizer))
julia> set_attribute(model, MOA.Algorithm(), MOA.EpsilonConstraint())
julia> optimize!(model)
```

Get X_E and Y_N

```
julia> for i in 1:result_count(model)
julia>     z_opt = [ objective_value(md; result = i)[k] for k=1:p ]
julia>     x_opt = value.(x; result = i)
julia> end
```

Example (3/3)

Plot Y_N

```
julia> using Plots
julia> Plots.scatter(
julia>  [value(z[1]; result = i) for i in 1:result_count(md)],
julia>  [-1 * value(z[2]; result = i) for i in 1:result_count(md)];
julia>  xlabel = "objective 1",
julia>  ylabel = "objective 2",
julia>  title = "objective space",
julia>  legend = false,
julia>  xlims = (0,25),
julia>  ylims = (0,25),
julia>  aspect_ratio=:equal,
julia> )
```

↳ https://github.com/xgandibleux/EURO_MCDM2026/blob/main/3.examples/2IPim.ipynb

Preliminary Conclusion

Key Takeaways...

You are now able to...

- identify existing software for multi-objective optimization
- formulate a multi-objective (linear) optimization model
- implement and solve it in *Julia* using *JuMP* and *MOA*
- extract and interpret X_E and Y_N produced by the solver
- choose an appropriate solution algorithm for a given problem

... use *Julia*, *JuMP* and *MOA* as a integrated workflow for multi-objective optimization

Next Steps...

Going further with JuMP:

- modify the model data, the model itself
- advanced modeling techniques, including sets and indices
- advanced resolution techniques, including callbacks to solvers
- other classes of optimization problems

Going further with MOA:

- compare the MOA algorithms with your own algorithm
- contribute by integrating your own algorithm into MOA

Going further with Julia:

- discover and learn to code in Julia
- use packages from among the 12,000 available packages

Références (1/2)...

JuMP (documentation, resources, community, etc.):

`https://jump.dev/`
`https://jump.dev/JuMP.jl/stable/`

Miles Lubin, Oscar Dowson, Joaquim Dias Garcia, Joey Huchette, Benoît Legat, Juan Pablo Vielma. JuMP 1.0: Recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*. 2023.

MultiObjectiveAlgorithms (source code, examples, tutorials):

`https://github.com/jump-dev/MultiObjectiveAlgorithms.jl`
`https://jump.dev/JuMP.jl/stable/tutorials/linear/multi_objective_examples/`

Oscar Dowson, Xavier Gandibleux, Gökhan Kof. MultiObjectiveAlgorithms.jl: a Julia package for solving multi-objective optimization problems. *INFORMS Journal of Computing*, 2026.

Xavier Gandibleux. Native Multi-Objective Mathematical Programming Software. Chapter 15 in *Handbook on Multi-objective Combinatorial Optimisation* (M. Ehrgott, J. Figueira, Ö. Karsu, L. Paquete Edts.). Springer, 2026 (to appear).

References (2/2)...

Books

Introduction to integer programming and applications with Julia
Luiz Antonio Nogueira Lorena, Luiz Henrique Nogueira Lorena, and Antônio Augusto Chaves
2026
https://intro-ilp-julia.github.io/book/

Julia Programming for Operations Research
Changhyun Kwon
2nd Edition. 260 pages. ISBN-10: 1798205475, ISBN-13: 9781798205471.
https://www.chkwon.net/julia/juliabook/juliabook2.html

Algorithms for Optimization
Mykel J. Kochenderfer, and Tim A. Wheeler
The MIT Press Cambridge, Massachusetts London, England
https://algorithmsbook.com/optimization/files/optimization.pdf